Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-14 — Página Web Metodologías TI

1. CONTEXTO
Se requiere una página web informativa/editorial sobre metodologías de informática y TI. El stack está explícitamente definido por el cliente: HTML5, CSS y JS vanilla. La página tiene 3 secciones de contenido (Home, Historias, Noticias) más FAQ, aviso de cookies y análisis de tráfico. No hay backend, no hay autenticación, no hay base de datos. El contenido es estático y editorial.

EVALUACIÓN DE ENTORNO CRÍTICA: El requerimiento especifica HTML5/CSS/JS puro sin mención de servidor Node/Python/etc. El artefacto final debe funcionar en entorno LOCAL (file:///) Y en servidor HTTP estático (Nginx/Apache/GitHub Pages). Por principio de menor denominador común y máxima portabilidad, se adopta la restricción más estricta: **PROHIBIDO usar ES6 modules (`type='module'`), import/export nativos o división de lógica JS en múltiples archivos con dependencias entre sí**. Todo el JavaScript irá consolidado en un único archivo `main.js` con patrón IIFE/namespace para evitar contaminación del scope global.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single-File-Group Static Site (Monolito de Presentación)
- 1 archivo HTML por sección lógica (SPA simulada con hash routing en JS)
- 1 archivo CSS consolidado
- 1 archivo JS consolidado
- Sin bundlers (Webpack/Vite/Rollup): YAGNI — no hay módulos que empaquetar
- Sin frameworks (React/Vue/Angular): YAGNI — contenido editorial estático no justifica 200KB+ de runtime
- Sin backend: el contenido se hardcodea en JS como arrays de objetos (data layer simulado)
- DOMPurify cargado via CDN con SRI hash para sanitización XSS
- Google Analytics via script tag estándar (GA4)

3. JUSTIFICACIÓN BASADA EN YAGNI/KISS
- **¿Por qué NO microservicios/API REST?** No hay datos dinámicos que persistan. El contenido de artículos y noticias es editorial y puede vivir como objetos JS. Agregar un backend Node/Express añadiría: servidor, proceso de deployment, CORS, mantenimiento de dependencias npm — todo para servir texto estático. Costo: alto. Beneficio: cero para este MVP.
- **¿Por qué NO React/Vue?** El criterio de carga <3 segundos favorece páginas sin hydration cost. Un SPA framework añade ~100-300ms de parse/execute JS en móviles de gama media. HTML semántico + CSS es más rápido y más accesible (WCAG 2.1 AA) por defecto.
- **¿Por qué NO múltiples archivos JS?** Compatibilidad file:/// y eliminación de problemas CORS en desarrollo local. Un solo archivo JS con namespace interno es igualmente mantenible para un proyecto de esta escala.
- **¿Por qué NO base de datos?** YAGNI absoluto. El contenido editorial de 3 secciones no requiere persistencia. Si en el futuro se necesita CMS, se puede migrar el array de objetos JS a una API JSON sin cambiar el frontend.
- **¿Por qué NO SSR/Next.js?** El stack está contractualmente definido como HTML5/CSS/JS. Además, SSR requiere servidor Node en runtime — incompatible con hosting estático.

4. DECISIONES DE SEGURIDAD (XSS/CSP)
- **DOMPurify**: Cargado via CDN con atributo `integrity` (SRI) para prevenir supply-chain attacks. Toda interpolación de contenido dinámico (búsquedas, comentarios futuros) pasa por `DOMPurify.sanitize()`.
- **CSP**: Meta tag `Content-Security-Policy` en el `<head>` restringiendo `script-src` a `'self'` + dominios de CDN

whitelisted (Google Analytics, DOMPurify CDN). Nota: CSP via meta tag no cubre todas las directivas (ej. `frame-ancestors`), pero es la única opción sin servidor HTTP que controle headers. Se documenta esta limitación.
- **Escape de caracteres**: Función utilitaria `escapeHTML()` en `main.js` para cualquier contenido que se inserte via `innerHTML`.
- **HTTPS**: Fuera del alcance del código (responsabilidad del hosting). Se documenta en README que el deploy DEBE ser sobre HTTPS.
- **Cookie Consent**: Implementado en JS puro con `localStorage` para persistir preferencia. Sin librerías externas (YAGNI).

5. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (BAJO):** El contenido hardcodeado en JS requiere un desarrollador para actualizar artículos. Mitigación futura: migrar el array de datos a un archivo `data.json` servido via fetch() cuando el entorno sea HTTP.
- **Riesgo 2 (MEDIO):** CSP via meta tag es menos robusto que CSP via HTTP header. Mitigación: documentar configuración de Nginx/Apache para headers en el README de deployment.
- **Riesgo 3 (BAJO):** Google Analytics requiere consentimiento de cookies (GDPR). Mitigación: el script de GA se inicializa SOLO después de que el usuario acepta cookies en el banner.
- **Riesgo 4 (BAJO):** Sin servidor, no hay HTTPS en desarrollo local. Mitigación: usar Live Server de VSCode o `python -m http.server` para desarrollo.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 2
- assets/js/main.js | Estado: APROBADO | Iteraciones: 1
- assets/css/styles.css | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 108,825
Tokens de salida (output): 39,472
Total tokens: 148,297
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~17.7 horas-hombre ahorradas
"H squad de 4 personas × 2.3 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“910 input | !‘1,733 output | 1 llamadas | ~1.5h ahorradas
MICROPLANIFICACION-DISEÑO: !“13,298 input | !‘9,500 output | 4 llamadas | ~5.7h ahorradas
FRONTEND-DESARROLLO: !“36,011 input | !‘20,764 output | 5 llamadas | ~5.3h ahorradas
test: !“58,606 input | !‘7,475 output | 6 llamadas | ~5.2h ahorradas